

# Apache Iceberg

## Flink v1.20 Connector

### 核心模块源码设计文档

基于 Apache Iceberg 1.9.2 版本源码分析

文档版本：v1.0 | 生成日期：2026-03-22

# 目录

---

## 第一章 模块概述与整体架构

## 第二章 核心流程图

- 2.1 Source 数据读取流程
- 2.2 Sink V1 数据写入流程
- 2.3 Sink V2 数据写入流程
- 2.4 Dynamic Sink 动态多表写入流程
- 2.5 Catalog 加载与表发现流程
- 2.6 Range Shuffle 数据分布流程
- 2.7 表维护任务调度流程

## 第三章 核心时序图

- 3.1 Source 批模式读取时序
- 3.2 Source 流模式读取时序
- 3.3 Sink Checkpoint 提交时序
- 3.4 Range Shuffle 统计协调时序

## 第四章 核心类设计说明

- 4.1 Catalog 与表管理层
- 4.2 Source 数据读取层
- 4.3 Sink 数据写入层
- 4.4 Range Shuffle 数据分布层
- 4.5 Dynamic Sink 动态写入层
- 4.6 Maintenance 表维护层
- 4.7 Data 数据读写与类型转换层

## 第五章 关键设计模式与最佳实践

# 第一章 模块概述与整体架构

## 1.1 模块简介

Apache Iceberg Flink v1.20 Connector 是 Iceberg 与 Apache Flink 1.20 版本的集成模块，提供了完整的数据湖读写能力。该模块包含约 260 个 Java 源文件，涵盖 Catalog 管理、数据源（Source）、数据汇（Sink）、数据格式读写（Parquet/ORC/Avro）、Range Shuffle 数据分布优化、动态多表写入（Dynamic Sink）以及自动化表维护（Maintenance）等完整功能。

## 1.2 包结构总览

| 包名                       | 类数量   | 核心职责                           |
|--------------------------|-------|--------------------------------|
| org.apache.iceberg.flink | 23    | Catalog、Schema 转换、类型映射、配置      |
| o.a.i.flink.actions      | 2     | 数据文件重写操作                       |
| o.a.i.flink.data         | 17    | Avro/ORC/Parquet 格式读写器         |
| o.a.i.flink.source       | 18+42 | FLIP-27 Source（枚举器+分配器+读取器+分片） |
| o.a.i.flink.sink         | 35    | Sink V1/V2 写入器与提交器             |
| o.a.i.flink.sink.shuffle | 28    | Range Shuffle 数据统计与分区          |
| o.a.i.flink.sink.dynamic | 30    | 动态多表写入                         |
| o.a.i.flink.maintenance  | 50    | 自动化表维护                         |
| o.a.i.flink.util         | 5     | 工具类                            |

## 1.3 整体架构图

Iceberg Flink v1.20 Connector 的整体架构分为五个核心层次：

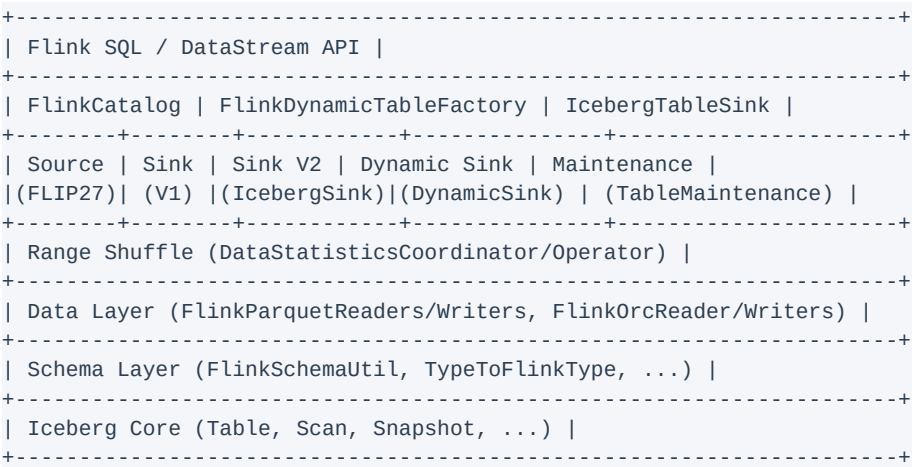


图 1-1：Iceberg Flink v1.20 Connector 整体架构



FlinkSink 是基于 Flink 旧版 API 的写入实现，使用传统的 Operator Chaining 方式手动编排算子链。整体拓扑为：数据分发 -> 并行写入 -> 单并行度提交 -> DiscardingSink。

```
+=====+
| FlinkSink (V1) Pipeline |
+=====+
| DataStream[RowData] |
| | |
| v |
| +-----+ |
| | distributeDataStream| [REDACTED]/Hash/Range [REDACTED] |
| | ([REDACTED]) | NONE / HASH / RANGE |
| | +-----+ |
| | |
| | v |
| | +-----+ parallelism = N |
| | |IcebergStreamWriter| [REDACTED] |
| | |([REDACTED]) | Checkpoint [REDACTED] flush |
| | | +-----+ |
| | | FlinkWriteResult (dataFiles + deleteFiles) |
| | | v |
| | | +-----+ parallelism = 1 |
| | | |IcebergFilesCommitter| [REDACTED] Writer [REDACTED] |
| | | |([REDACTED]) | Checkpoint [REDACTED] Iceberg |
| | | | +-----+ |
| | | | |
| | | | v |
| | | | +-----+ |
| | | | | DiscardingSink | [REDACTED] |
| | | | +-----+ |
+=====+
```

图 2-2 : FlinkSink V1 写入流程

## 2.3 Sink V2 数据写入流程 (IcebergSink)

IcebergSink 基于 Flink Sink V2 框架，利用 PreWrite/Writer/PreCommit/Committer/PostCommit 五阶段流水线。相比 V1，V2 的状态管理由框架自动完成，且支持 PostCommit 维护拓扑。

```
+=====+
| IcebergSink (V2) Pipeline |
+=====+
| DataStream[RowData] |
| | SupportsPreWriteTopology |
| v |
| +-----+ |
| | addPreWriteTopology | █████(NONE/HASH/RANGE) |
| +-----+ |
| | SinkWriter |
| v |
| +-----+ parallelism = N |
| | IcebergSinkWriter | █████, prepareCommit █ flush |
| +-----+ |
| | WriteResult |
| v SupportsPreCommitTopology |
| +-----+ parallelism = 1 |
| | IcebergWriteAggregat. | █ WriteResult |
| | | █ IcebergCommittable |
| +-----+ |
| | SupportsCommitter |
| v |
| +-----+ parallelism = 1 |
| | IcebergCommitter | █ Iceberg (AppendFiles/RowDelta) |
| +-----+ |
| | SupportsPostCommitTopology |
| v |
| +-----+ |
| | PostCommit ████ | █████ / █████ / ████ |
| +-----+ |
+=====+
```

图 2-3 : IcebergSink V2 写入流程

## 2.4 Dynamic Sink 动态多表写入流程

DynamicIcebergSink 支持同时写入任意数量的 Iceberg 表，根据用户路由逻辑动态选择目标表。

```
DataStream[RowData]
|
v
+-----+
| DynamicRecordGenerator | █████Schema█████
+-----+
| DynamicRecord
v
+-----+
| DynamicTableUpdateOperator | █████ / Schema █
+-----+
| DynamicRecordInternal
v
+-----+ KeyBy(writerKey) parallelism = N
| DynamicWriter | █ WriteTarget █████ TaskWriter
+-----+
| DynamicWriteResult
v
```

```

+-----+ KeyBy(tableName) parallelism = 1
| DynamicWriteResultAggr. | ████████
+-----+
| DynamicCommittable
v
+-----+ parallelism = 1
| DynamicCommitter | ██████ Iceberg █
+-----+

```

图 2-4 : Dynamic Sink 动态多表写入流程

## 2.5 Catalog 加载与表发现流程

Iceberg 的 Catalog 和 Table 加载采用序列化加载器模式，解决了 Flink 分布式场景下对象不可序列化的问题。



图 2-5 : Catalog 加载流程

## 2.6 Range Shuffle 数据分布流程

Range Shuffle 通过收集数据分布统计信息来优化写入时的数据聚簇性，减少小文件数量。支持 Map（低基数精确统计）和 Sketch（高基数近似统计）两种模式。



图 2-6 : Range Shuffle 数据分布流程



## 2.7 表维护任务调度流程

TableMaintenance 框架提供自动化表维护能力，支持快照过期、孤儿文件清理和数据文件重写。

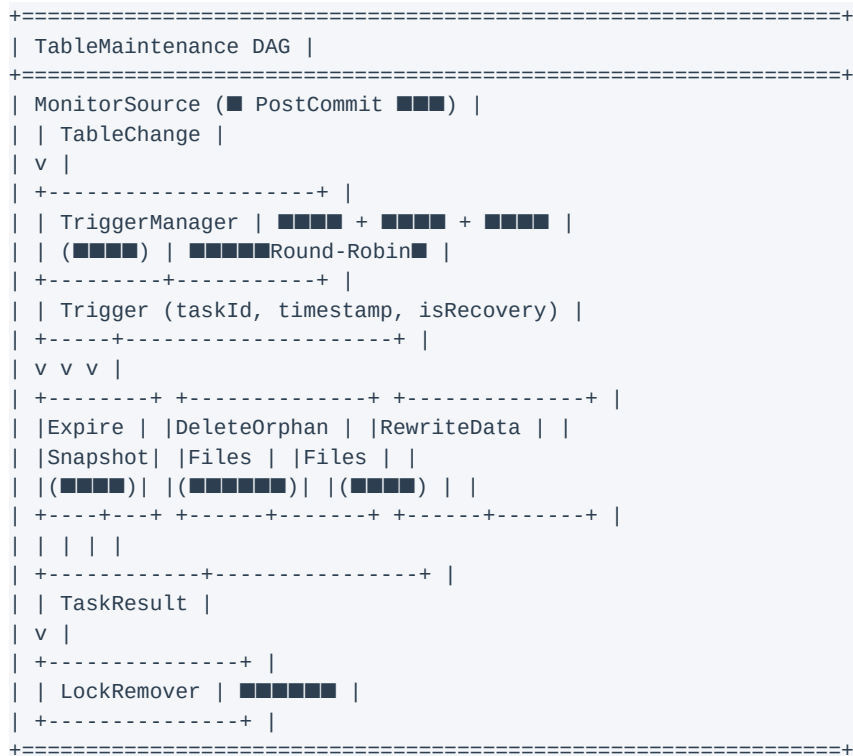


图 2-7：表维护任务调度流程

## 第三章 核心时序图

### 3.1 Source 批模式读取时序

批模式下，Split 在 Source 构建时提前规划完成，StaticIcebergEnumerator 一次性分配。

```
IcebergSource FlinkSplitPlanner StaticEnumerator SourceReader
| | | |
|-- build() ----->| | |
| |-- planSplits() -->| | |
| | |-- planTasks() | |
| | |-- Iceberg Scan | |
| |<-- Split[] -----| | |
|<-- Source -----| | |
| | | |
| Flink Runtime ■■ | | |
| |-- createEnumerator() ----->| |
| | (■■■■■■splits) | |
| |-- createReader() ----->|
| | | |
| | | |<-- requestSplit|
| | | |-- assignSplit>|
| | | | |-- fetch()
| | | | |-- ■■■■
| | | |<-- splitDone |
| | | |-- assignSplit>|
| | | ... ■■ ... |
| | | |-- noMoreSplit>|
```

图 3-1 : Source 批模式读取时序

### 3.2 Source 流模式读取时序

流模式下，ContinuousIcebergEnumerator 周期性调用 discoverSplits() 发现增量 Split。

```
ContinuousEnumerator SplitPlanner SplitAssigner SourceReader Iceberg Table
| | | |
|-- start() | | | |
| (■■■■■■) | | | |
| | | |
|-- discoverSplits() -->| | |
| (■■IO■■) |-- planSplits() ----->|
| | | | IncrAppendScan
| |<-- ContinuousEnumerationResult -----|
|<-- processResult() --| | |
| | | |
|-- CAS■■position | | | |
|-- onDiscoveredSplits() ----->| | |
| | |-- ■■■■ -->| |
| | | |-- fetch() -->|
| | | |<-- ■■ -----|
| | | |
| (monitorInterval■) | | | |
|-- discoverSplits() -->| ... ■■ .. | | |
```

图 3-2 : Source 流模式读取时序

### 3.3 Sink Checkpoint 提交时序

以下展示 Sink V1 (IcebergFilesCommitter) 在 Checkpoint 过程中的文件提交流程。

```
StreamWriter[0] StreamWriter[1] FilesCommitter Iceberg Table Flink Checkpoint
| | | |
|-- write() ----->| | |
| ... |-- write() ---->| | |
| | | | +-----+
| | | | |CK ■■ |
| | | | +---+---+
|-- flush() ----->| | |
| complete() |-- flush() --->| | |
| new Writer | complete() | | |
| | new Writer | | |
|-- emit(WR) ----->| | |
| |-- emit(WR) ->| | |
| | |-- snapshotState()| | |
| | | ■DeltaManifest | | |
| | | |
| | | | notifyCheckpointComplete()
| | | |<-----|
| | | |
| | |-- commitUpToCheckpoint() |
| | |-- ■DeltaManifests |
| | |-- AppendFiles/RowDelta -->|
| | |-- commit()
| | |<-- ■■■■ -----| |
```

图 3-3 : Sink Checkpoint 提交时序

### 3.4 Range Shuffle 统计协调时序

```
Operator[0] Operator[1] Coordinator Partitioner Flink CK
| | | |
|-- processEl() | | |
| ■■■■■■ |-- processEl()| | |
| | ■■■■■■| | |
| | | | +-----+
| | | | |CK ■■ |
| | | | +---+---+
|-- snapshotSt() | | | |
| ■■■■■■ =====>| | |
| |-- snapshot() | | |
| | ■■■■ ==>| | |
| | | |
| | |-- ■■■■■■ | |
| | |-- ■■Global | |
| | | (Map: ■■■■)| |
| | | (Sketch: ■■)| |
| | | |
|<== ■■GlobalStatistics ===== | | |
| |<== ■■ ===== | | |
| | | |
|-- ■■■■ =====>| |
| |-- ■■■■ =====>| |
| | | |-- ■■■■
```

图 3-4 : Range Shuffle 统计协调时序

# 第四章 核心类设计说明

## 4.1 Catalog 与表管理层

### 4.1.1 FlinkCatalog

FlinkCatalog 是 Flink AbstractCatalog 的实现，作为 Iceberg Catalog 在 Flink 中的适配器。Flink 的 database 映射为 Iceberg namespace 的最后一级，通过 baseNamespace 前缀支持多级命名空间。支持 CachingCatalog 缓存包装以提升性能。

| 关键字段           | 类型            | 作用                         |
|----------------|---------------|----------------------------|
| catalogLoader  | CatalogLoader | 可序列化 Catalog 加载器           |
| icebergCatalog | Catalog       | 底层 Iceberg Catalog（可被缓存包装） |
| baseNamespace  | Namespace     | 命名空间前缀                     |
| cacheEnabled   | boolean       | 是否启用缓存                     |

核心方法：toldentifier() 转换 Flink ObjectPath 为 Iceberg TableIdentifier；getTable() 加载表并转为 CatalogTable；alterTable() 支持 TableChange 的 Schema 变更。

### 4.1.2 CatalogLoader

可序列化的 Catalog 加载器接口。4 种实现：HiveCatalogLoader、HadoopCatalogLoader、RESCatalogLoader、CustomCatalogLoader（反射加载自定义实现）。

### 4.1.3 TableLoader

可序列化的 Table 加载器。2 种实现：HadoopTableLoader（文件路径直接加载）和 CatalogTableLoader（先加载 Catalog 再加载 Table）。

### 4.1.4 FlinkDynamicTableFactory

Flink SPI 的 DynamicTableSourceFactory/DynamicTableSinkFactory 实现，factoryIdentifier = "iceberg"。支持 Iceberg Catalog 内使用和非 Iceberg Catalog 场景（自动建库建表）。

## 4.2 Source 数据读取层

### 4.2.1 IcebergSource

FLIP-27 新 Source 的顶层入口，实现 Source 接口。协调 SplitEnumerator、SourceReader、序列化器。支持批处理和流处理。Builder 提供丰富的链式 API。

| 关键字段            | 说明                        |
|-----------------|---------------------------|
| tableLoader     | 表加载器（实际操作需 clone）         |
| scanContext     | 封装全部扫描参数                  |
| readerFunction  | Split -> 数据记录的读取函数        |
| assignerFactory | Split 分配器工厂               |
| splitComparator | Split 排序比较器（watermark 场景） |
| emitter         | 记录发射器（可附带 watermark）      |
| batchSplits     | 批模式下的 Split 缓存（volatile）  |

### 4.2.2 ScanContext

不可变的可序列化配置容器。封装：快照选择（snapshotId/branch/tag）、增量范围、流式策略、Split 控制、投影过滤、watermark 配置等全部扫描参数。

### 4.2.3 FlinkSplitPlanner

无状态工具类，Iceberg Scan API 与 Flink Source Split 的桥梁。BATCH 模式用 table.newScan()，INCREMENTAL 模式用 table.newIncrementalAppendScan()。

### 4.2.4 Enumerator 枚举器

| 类名                          | 模式  | 核心行为                               |
|-----------------------------|-----|------------------------------------|
| StaticIcebergEnumerator     | 批模式 | 一次性接收所有 Split，不再发现新的               |
| ContinuousIcebergEnumerator | 流模式 | 周期性 discoverSplits()，CAS 位置管理，内置节流 |

### 4.2.5 IcebergSourceReader

继承 SingleThreadMultiplexSourceReaderBase，多线程多路复用模型。通过 SplitRequestEvent 向 Enumerator 请求 Split。IcebergSourceSplitReader 管理 Split 队列，逐个顺序读取。

### 4.2.6 DefaultSplitAssigner

默认分配策略。无序模式 ArrayDeque（FIFO），有序模式 PriorityQueue。通过 CompletableFuture 实现异步等待机制。

## 4.3 Sink 数据写入层

### 4.3.1 Sink V1 与 V2 对比

| 特性         | FlinkSink (V1)          | IcebergSink (V2)         |
|------------|-------------------------|--------------------------|
| API 基础     | AbstractStreamOperator  | Flink Sink V2 框架         |
| 状态管理       | 手动管理 checkpoint 状态      | 框架自动管理                   |
| 写入器        | IcebergStreamWriter     | IcebergSinkWriter        |
| 提交器        | IcebergFilesCommitter   | IcebergCommitter         |
| PostCommit | 不支持                     | 支持维护拓扑                   |
| 拓扑编排       | 手动 transform + chaining | 框架 addPreWriteTopology 等 |

### 4.3.2 IcebergStreamWriter (V1)

并行写入算子。prepareSnapshotPreBarrier() 触发 flush，关闭文件并创建新 TaskWriter，WriteResult 包装为 FlinkWriteResult 发送下游。

### 4.3.3 IcebergFilesCommitter (V1)

单并行度提交算子。NavigableMap 按 checkpointId 缓存 DeltaManifests。notifyCheckpointComplete 时提交：无 delete 用 AppendFiles，有 delete 用 RowDelta。

### 4.3.4 IcebergSinkWriter (V2)

V2 写入器，实现 CommittingSinkWriter。write() 委托 TaskWriter，prepareCommit() 调用 complete() 完成文件并返回 WriteResult。

### 4.3.5 IcebergCommitter (V2)

V2 提交器。按 checkpointId 排序处理 CommitRequest，通过 flink.max-committed-checkpoint-id 实现幂等提交。

### 4.3.6 RowDataTaskWriterFactory

连接 Sink 与底层 Iceberg 写入器的核心枢纽。根据四种场景创建不同 TaskWriter：

| 场景                | 等值删除字段 | 分区 | Writer 类型                |
|-------------------|--------|----|--------------------------|
| 仅 INSERT，无分区      | 否      | 否  | UnpartitionedWriter      |
| 仅 INSERT，有分区      | 否      | 是  | PartitionedFanoutWriter  |
| INSERT+DELETE，无分区 | 是      | 否  | UnpartitionedDeltaWriter |
| INSERT+DELETE，有分区 | 是      | 是  | PartitionedDeltaWriter   |

### 4.3.7 BaseDeltaTaskWriter

CDC/UPSERT                  写入器基类。按                  RowKind                  分发：INSERT/UPDATE\_AFTER  
写数据文件；UPDATE\_BEFORE/DELETE 写 equality-delete 文件。UPSERT 模式下 UPDATE\_BEFORE  
被忽略，UPDATE\_AFTER 先删旧键再写新行。

## 4.4 Range Shuffle 数据分布层

### 4.4.1 DataStatisticsCoordinator

JM 端 OperatorCoordinator。接收子任务统计，聚合生成 GlobalStatistics 并广播。Map 模式：贪心装箱算法分配 key 到子任务；Sketch 模式：计算范围边界。

### 4.4.2 DataStatisticsOperator

TM 端算子。提取 SortKey 更新本地统计，checkpoint 时上报。支持 Map 到 Sketch 自动迁移（高基数场景）。UnionListState 持久化全局统计。

### 4.4.3 分区器对比

| 特性     | MapRangePartitioner | SketchRangePartitioner |
|--------|---------------------|------------------------|
| 适用场景   | 低基数                 | 高基数                    |
| 统计方式   | 精确频率计数              | 近似采样（Sketch）           |
| 分区策略   | 查 MapAssignment     | rangeBounds 二分查找       |
| 未知 key | Round-Robin 回退      | 自然落入范围区间               |
| 内存开销   | 与 key 数成正比          | 固定大小                   |



## 4.5 Dynamic Sink 动态写入层

### 4.5.1 DynamicIcebergSink

动态多表写入的 Sink V2 入口。通过 DynamicRecordGenerator 将数据转换为带目标表信息的 DynamicRecordInternal，支持同时写入多表，自动建表和 Schema 演化。

### 4.5.2 DynamicWriter

为每个 WriteTarget（表名+分支+schema+spec+upsert模式）维护独立 TaskWriter。LRU 缓存管理 RowDataTaskWriterFactory。Checkpoint 时完成所有 Writer。

### 4.5.3 DynamicCommitter

动态多表提交器。按 TableKey 和 checkpointId 分组，通过 snapshot Summary 中的 checkpoint-id 实现幂等提交，内部使用 MaxCommittedCheckpointIdValidator。

### 4.5.4 EvolveSchemaVisitor

Schema 自动演化核心。对比现有和目标 Schema，支持：添加新列、拓宽类型、调整列顺序、删除不使用的列（可选）。不支持列重命名。

## 4.6 Maintenance 表维护层

### 4.6.1 TableMaintenance

顶层编排入口。Builder 模式组装 DAG：变更监控 -> 触发管理 -> 任务执行 -> 锁释放。支持独立作业模式（forTable）和嵌入模式（forChangeStream）。

### 4.6.2 MonitorSource

无界源，周期性轮询快照变更。沿 parentId 链回溯，跳过 REPLACE 操作。RateLimiter 控制频率。

### 4.6.3 TriggerManager

调度决策核心。累积 TableChange，TriggerEvaluator 判断触发，分布式锁防并发。Round-Robin 公平调度防止任务饥饿。

### 4.6.4 三种维护任务

| 维护任务              | 核心功能             | 执行阶段                                                                      |
|-------------------|------------------|---------------------------------------------------------------------------|
| ExpireSnapshots   | 删除过期快照及文件        | Processor -> DeleteFiles                                                  |
| DeleteOrphanFiles | 检测并删除孤儿文件        | MetadataPlanner -> FileNameReader -> ListFiles -> Detector -> DeleteFiles |
| RewriteDataFiles  | 合并小文件 Compaction | Planner -> Runner -> Committer                                            |

## 4.7 Data 数据读写与类型转换层

### 4.7.1 Schema 转换

FlinkSchemaUtil 是双向转换门面，底层委托 FlinkTypeToType (Flink->Iceberg) 和 TypeToFlinkType (Iceberg->Flink)。类型映射不是 1:1 精确对应。

| Flink 类型                 | Iceberg 类型                  | 说明             |
|--------------------------|-----------------------------|----------------|
| VarCharType              | StringType                  | 长度信息丢失         |
| TinyIntType/SmallIntType | IntegerType                 | 统一为 int        |
| BinaryType(n)            | FixedType(n)                | 定长二进制          |
| TimestampType            | TimestampType.withoutZone() | 微秒精度           |
| LocalZonedTimestampType  | TimestampType.withZone()    | 带时区            |
| ArrayType                | ListType                    | 保留 nullability |
| RowType                  | StructType                  | 递归处理子字段        |

### 4.7.2 Parquet 读写器

FlinkParquetReaders/Writers 采用访问者模式遍历 Schema 和 MessageType，为每种类型构建 Reader/Writer。支持 Decimal、Timestamp 多精度、复杂嵌套类型。内部使用 ReusableArrayData/ReusableMapData 减少 GC。

### 4.7.3 ORC 读写器

FlinkOrcReader 通过 OrcSchemaWithTypeVisitor 构建 Reader 树。FlinkOrcWriters 直接操作 ColumnVector，支持 3 倍动态向量扩容。

### 4.7.4 RowDataProjection

RowData 列投影包装器。按目标 Schema 选取部分列，STRUCT 递归创建嵌套 RowDataProjection。不支持 List/Map 内嵌套子元素部分投影。

# 第五章 关键设计模式与最佳实践

## 5.1 核心设计模式

### 序列化加载器模式

CatalogLoader/TableLoader 都是可序列化接口。将"如何创建"序列化传输到 TaskManager，在远端按需重建对象。这是整个模块最基础的设计模式。

### Builder 模式

几乎所有核心类都采用 Builder 模式：IcebergSource.Builder、IcebergSink.Builder、ScanContext.Builder 等。统一收集配置并执行校验。

### 访问者模式 (Visitor Pattern)

数据层大量使用：FlinkParquetReaders.ReadBuilder、FlinkTypeToType、TypeToFlinkType、EvolveSchemaVisitor 等。遍历类型树为每种类型构建处理器。

### Coordinator-Operator 协调

Range Shuffle 使用 Coordinator (JM) + Operator (TM)。Operator 收集本地统计上报，Coordinator 聚合后广播全局统计。

### 精确一次语义

Writer 在 Checkpoint barrier 前 flush，Committer 在 notifyCheckpointComplete 提交。V2 通过 flink.max-committed-checkpoint-id 实现幂等提交。

### CAS 位置管理

ContinuousIcebergEnumerator 使用 AtomicReference + CAS 管理 position，确保异步 IO 线程和协调器线程之间安全更新。

## 5.2 关键配置参数

| 参数                                 | 默认值     | 说明                     |
|------------------------------------|---------|------------------------|
| write.distribution-mode            | none    | 数据分布模式：none/hash/range |
| write.upsert.enabled               | false   | 是否启用 UPSERT 模式         |
| write.target-file-size-bytes       | 128MB   | 目标数据文件大小               |
| write.format.default               | parquet | 默认文件格式                 |
| flink.max-continuous-empty-commits | 10      | 最大连续空提交次数              |
| source.monitor-interval            | 60s     | 流模式 Split 发现间隔         |

| 参数                | 默认值   | 说明         |
|-------------------|-------|------------|
| source.split-size | 128MB | Split 目标大小 |

### 5.3 扩展性设计

模块具备良好的扩展性：(1) CatalogLoader/TableLoader 策略模式支持新 Catalog 类型；(2) DynamicRecordGenerator 接口支持自定义多表路由逻辑；(3) SplitAssignerFactory 接口支持自定义 Split 分配策略；(4) MaintenanceTaskBuilder 基类支持自定义维护任务；(5) Range Shuffle 的 Map/Sketch 自动切换适应不同数据分布。

— 文档结束 —

本文档基于 Apache Iceberg 1.9.2 版本 Flink v1.20 Connector 模块源码分析生成。